

Microcontroladores

Roteiro 1 — Bits e máscaras

Raoni F. S. Teixeira · Rodolfo Quadros · DENE/UFMT · 1 sessão · 1,0 ponto · grupos de 3

Objetivos desta sessão

Ao final desta sessão o grupo deve ser capaz de:

1. Operar bits individuais de um registrador sem alterar os vizinhos;
2. Usar os operadores &, |, ^, ~, << e >> com propósito definido;
3. Explicar o que é uma máscara de bits e por que ela existe;
4. Justificar por que os LEDs do kit acendem com nível lógico baixo.

1 Por que isso é conteúdo, e não revisão

Disciplinas de Algoritmos trabalham com **valores**: somar, comparar, atribuir. Aqui o programa conversa com **registradores**, onde cada bit comanda um pino físico diferente.

Escrever `TRISC = 0x00` configura os oito pinos de uma vez — inclusive os sete que você não queria tocar. Em um sistema com atuador ligado a um deles, isso é um acidente, não um detalhe de estilo.

Conceito central

`TRISC &= ~0b00000110` é a forma correta de mexer em dois pinos preservando os outros seis.

Entender essa linha é programação de microcontrolador — não é pré-requisito dela. Máscara de bits é o vocabulário nativo da disciplina.

2 Configuração de bancada

Configuração de bancada

Chaves SWITCHS (PORTB)	Todas em OFF
CH2-1 (LCD)	OFF
CH3-3 e CH3-5	OFF
Demais chaves	OFF

Nesta sessão os únicos periféricos usados são os 8 LEDs do PORTD.

3 Fundamento: o LED que acende com zero

Os 8 LEDs do kit são **ativos em nível baixo**: o pino em 0 acende o LED, o pino em 1 apaga. O anodo está ligado ao +5 V e o catodo ao pino do PIC, que funciona como caminho para o terra.

⚠ Atenção

Consequência prática: para **acender** o LED 0 você escreve **0** no bit 0. Para acender todos, LATD = 0x00.

Isso é comum em hardware real e é a origem de metade dos enganos de quem começa. Anote no caderno agora.

4 Parte 1 — Escrita direta

Tarefa

Compile e grave um programa que faça apenas:

```
LATD = 0b11110000;  
TRISD = 0x00;
```

1.1 Quais LEDs acendem? Desenhe os oito círculos e marque os acesos.

1.2 O resultado bate com sua expectativa? Se não, o que você esperava e por quê?

5 Parte 2 — As quatro operações fundamentais

📎 Previsão — registre ANTES de gravar

Para cada operação desta seção, escreva o resultado esperado **antes** de gravar. Depois verifique.

Considere sempre o estado inicial LATD = 0xFF — todos os LEDs apagados.

5.1 Ligar um bit: OU com máscara

```
LATD |= 0b00000001;
```

Previsão: _____ Observado: _____

5.2 Desligar um bit: E com máscara invertida

```
LATD &= ~0b00000001;
```

Previsão: _____ Observado: _____

5.3 Inverter um bit: OU-exclusivo

```
LATD ^= 0b00000001;
```

Aplique duas vezes seguidas.

2.3 O que acontece após a segunda aplicação? Por quê?

5.4 Testar um bit

```
if ((PORTD & 0b00000001) == 0) {
    /* o bit 0 está em nível baixo */
}
```

2.4 Por que a comparação é com 0 e não com 1?

6 Parte 3 — Construção de máscara por deslocamento

Escrever `0b00001000` funciona, mas não é reaproveitável. A forma usual é **construir** a máscara a partir do número do bit:

```
#define BIT(n) (1u << (n))
```

Tarefa

3.1 — Complete a tabela.

Expressão	Binário	Hexadecimal
BIT(0)		
BIT(3)		
BIT(7)		
BIT(2) BIT(5)		
~BIT(0)		

Tarefa

3.2 — Escreva as três macros usando `BIT(n)`. Lembre que os LEDs são ativos em nível baixo: **acender** significa escrever zero.

```
#define LED_ACENDER(n)    /* completar */  
#define LED_APAGAR(n)    /* completar */  
#define LED_INVERTER(n)  /* completar */
```

Tarefa

3.3 — Grave um programa que acenda **apenas** os LEDs 1, 3 e 6, usando suas macros.

Não é permitido escrever o byte inteiro de uma vez.

7 Parte 4 — Por que a máscara importa

Considere o kit XM118, onde:

- RC1 aciona a **resistência de aquecimento**;
- RC2 aciona a **ventoinha**;
- RC6 e RC7 são a **comunicação serial**.

 Previsão — registre ANTES de gravar

4.1 — Um colega precisa ligar a ventoinha e escreve:

```
LATC = 0b00000100;
```

O que acontece com a comunicação serial? E com a resistência de aquecimento?

4.2 Reescreva a linha de forma segura.

4.3 Em uma frase — qual a diferença entre `=` e `|=` quando o registrador comanda dispositivos independentes?

Conceito central

Um registrador de porta não é uma variável comum: ele é um **painel de comandos compartilhado**. Escrever nele por atribuição direta equivale a redefinir o estado de todos os dispositivos ligados àquela porta, e não apenas do que interessa.

8 Teste de aceitação

- ☐ LEDs 1, 3 e 6 acesos, usando macros construídas com `BIT(n)`.
- ☐ Um membro do grupo, sorteado pelo professor, explica por que `LATD &= ~BIT(2)` **apaga** o LED 2 em vez de acender.

9 Entrega e critério

Peso	Item
0,2	Teste de aceitação aprovado
0,2	Tabela 3.1 completa e correta
0,2	Macros 3.2 corretas para a lógica invertida
0,4	Respostas da Parte 4, com justificativa

Tabela 1: Distribuição do ponto do Roteiro 1.

Observação

Nas previsões da Parte 2, avalia-se o **registro da previsão**, não o acerto.

Previsão errada com raciocínio explícito vale integral. Previsão preenchida depois de observar o resultado não vale nada — e é facilmente identificável.

10 Armadilhas frequentes

Sintoma	Causa provável
LEDs invertidos do esperado	Esqueceu que são ativos em nível baixo
<code>LATD &= BIT(2)</code> apaga tudo	Faltou o <code>~</code> : a máscara precisa ser invertida
Nada acende	TRISD não foi zerado; pinos ainda são entrada
Um LED some ao mexer em outro	Usou <code>=</code> no lugar de <code> =</code> ou <code>&=</code>

Tabela 2: Diagnóstico rápido do Roteiro 1.

11 Para a próxima sessão

Tarefa

Escreva, sem compilar, um trecho que faça o LED 0 piscar.

Você ainda não viu laços em C nesta disciplina — use o que lembra de Algoritmos e não se preocupe com a sintaxe exata.